Anchor page 1

How to Plan a Software Project When You Are Not Technical

If you are thinking about commissioning a software build — whether that is a new app, a system to replace something your business already uses, or a tool to solve a problem you keep running into — the planning stage is where everything starts. And for most people without a technical background, it is also where things start to feel overwhelming.

Over the past two decades working on software projects — including large-scale platforms and early-stage builds — I've seen the same planning problems appear again and again. When the business problem, users, and scope are unclear at the start, development quickly becomes guesswork.

There is a lot of advice out there aimed at developers and product managers. This is not that. This is for the business owner who needs a new system and is not sure where to begin, and for the individual who has an idea for an app and wants to understand what they are actually getting into.

You do not need to become technical to plan this well. You just need to get clear on a few important things before you start talking to anyone who wants to build it for you.

Key Takeaways

- ✓ Planning is the most important stage of a software project — and the one most often rushed.
- ✓ Most projects run into trouble because of unclear requirements, not bad developers.
- ✓ Business owners upgrading existing systems need to map what they already have before briefing anyone.
- ✓ You should be able to describe what you are building, who it is for, and how you will measure success before approaching a developer.
- ✓ A written brief — even a short one — significantly improves the quality of quotes you receive.
- ✓ Getting clear on constraints early (budget, timeline, integrations) prevents the most common and costly surprises.

Why so many projects run into trouble early

Most software projects that go wrong do not fail because of bad developers. They run into trouble because not enough thinking happened before development started.

That is not a criticism — it is just how it tends to happen. Someone has an idea, or a problem that needs solving. They find a developer or an agency. They have a conversation. A quote comes back. Work begins. And somewhere along the way it becomes clear that what is being built is not quite what was imagined, or costs twice what was expected, or takes far longer than anyone said.

The gap between what you picture and what gets built is almost always a planning gap. Not a technical one.

There is also a specific trap that business owners fall into when upgrading an existing system. The temptation is to ask for what you already have, but better. The problem is that what you currently have has usually grown through

years of small workarounds and staff habits — and recreating it without thinking clearly about what you actually need just rebuilds the problems you were trying to leave behind.

> **Worth knowing**
>
> The conversations you have before a developer starts work are the most important conversations in the whole project. Getting those right — being clear about what you need, why you need it, and what success looks like — makes everything that follows easier and less expensive.

What you need to get clear on before you start

You do not need a lengthy written specification before you approach a developer. But you do need clear answers to a handful of questions. Without them, any quote you receive will be based on assumptions — and assumptions are where cost and timeline surprises come from.

What problem are you actually solving?

This sounds obvious, but it is worth sitting with. What is the specific thing that is broken, slow, or missing right now? Who does it affect? How often? The more precisely you can describe the problem, the better placed you are to judge whether what a developer proposes will actually solve it.

Who will use it?

Not a category of people — a real person, or a clearly described type of person, with a specific need. For business owners, this is often your own staff. That matters, because a system your team does not understand or trust will not get used, however well it is built.

What does it need to do first?

Not eventually — first. What is the smallest, most focused version of this that would be genuinely useful? This is the foundation of a well-scoped project. Everything else can come later. Getting clear on the first version stops the scope from growing out of control before build has even begun.

How will you know it is working?

What does success look like 90 days after launch? Pick one measure. It could be time saved, errors reduced, customers served, or something else entirely — but having a clear answer to this question keeps the whole project honest.

What are the constraints?

Budget. Timeline. Any existing systems the new build needs to work alongside. Any legal or data requirements. Anything that could limit your options. Bringing these into the conversation early means you are not discovering them halfway through.

The five things to settle before you speak to a developer

Think of these as the five things you need in place before any scoping conversation. They do not need to be formal documents — a page of clear notes is fine. But without them, you are effectively asking a developer to make your decisions for you.

1. A one-sentence description of what the product does. If you cannot explain it simply, the brief will be unclear — and developers will fill the gaps with their own assumptions, not yours.

2. A clear picture of who uses it and what they need it to do. For business owners, think about your staff as well as any customers. For individuals commissioning an app, think about the first real person who would find this useful.
3. A short list of what the first version must include — and a separate note of what can wait. This is your scope. Keeping it focused at the start is one of the most valuable things you can do for your budget and your timeline.
4. A plan for what happens after launch. Who looks after the system? Who handles it if something goes wrong? Who decides what gets built next? These questions are much easier to answer before you start than after.
5. A clear definition of done. How will you know the project has been delivered successfully? This needs to be agreed with your developer before work begins — not during the final week.

If you are replacing an existing system, there is one more thing

- Map what you currently have before you brief anyone. What does your existing system or process actually do? Where are the gaps and workarounds? What do people rely on day to day, even if it is clunky? Your developer cannot build a good replacement for something they do not understand — and you cannot brief one well without this clarity.
- Pay particular attention to data. Old systems often hold years of records. Moving that data to a new system is almost always more complex than it looks, and it needs to be planned for — not discovered as a problem in the final week of the project.

A simple planning framework

Use this table to organise your thinking before any conversation with a developer or agency. Work through each row and write plain answers — not a polished presentation, just honest notes. A founder or business owner who can fill this in clearly is genuinely ready to commission a build. If certain rows are hard to answer, that is useful to know now rather than later.

Area	What to think through
The problem	What is broken, missing, or not working well enough? Who does it affect and how often?
Current state	If replacing something existing — what does it do now? Where are the gaps and workarounds?
The user	Who will use this? What do they need it to do? How will you get their buy-in?
First version scope	What must the new system do at launch? What can be saved for a later phase?
Success measure	How will you know it is working 90 days after launch?
Constraints	Budget, timeline, existing systems to connect with, data or compliance requirements
After launch	Who owns, manages, and develops the system once it goes live?

When a bit of outside help makes sense

Some people can work through this on their own without too much difficulty. Others find it genuinely useful to have someone alongside them who has been through this process many times and can ask the questions they have not thought to ask yet.

An independent advisor — someone who is not the developer who will be building your product — can help you in ways that are hard to replicate on your own. They can look at what you are planning and tell you honestly whether it makes sense. They can help you write a brief that gets you accurate quotes rather than vague estimates. They can review what a developer proposes, so you are not trying to evaluate something technical with no frame of reference.

The most important word there is independent. A developer has a commercial interest in the project going ahead. An advisor whose only job is to help you make a good decision has a very different kind of interest — and that difference is worth a lot when you are about to commit a significant amount of money.

Not sure where to start with your planning?

Qube Catalyst works with business owners and founders at the very start of a software project — helping you get clear on what you need, what it should cost, and what to watch out for. No jargon, no pressure. Just honest guidance from someone who has seen a lot of these projects from the outside.

Learn more about independent product support from Qube Catalyst. **Book a call** or **Email me directly**

A quick sense-check before you approach anyone

Before you have a scoping conversation with a developer or agency, it is worth being able to answer most of these honestly.

- I can describe what I am building — or replacing — in one or two sentences
- I know who the primary users are and what they need it to do
- I have thought through what the first version needs to include, and what can wait
- I have a rough idea of my budget and timeline
- I know what existing systems this needs to work alongside, if any
- I have thought about data — what needs to move, and what might be lost or left behind
- I have a sense of what success looks like after launch
- I have written at least a short brief, even if it is just a page of notes

If most of these feel comfortable, you are in a good position to start having conversations. If several feel uncertain, spending a bit more time on the planning now will save you a lot of time and money later. That is not a criticism — it is just how these projects work. The thinking you do before you start is always the best investment you make.

A reassuring thought

Nobody who commissions a software build for the first time feels fully prepared. The goal is not to know everything before you begin — it is to be clear enough that you can have honest conversations, ask sensible questions, and make good decisions as the project unfolds. That is entirely achievable, and this is what good planning gives you.

Related reading

- How to Choose a Development Agency
- What a Realistic MVP Scope Looks Like
- How to Track Progress on a Software Build

Qube Catalyst helps business owners and founders plan, commission, and review software builds — independently of the developers doing the work.

Anchor page 2

How to Choose a Development Agency

Choosing a development agency is one of the most important decisions you will make in a software project. After working on software projects for more than two decades — both inside large technology organisations and alongside founders building their first platforms — I've seen how often agency selection determines whether a project succeeds or struggles. Get it right and you have a team that builds what you need, keeps you informed, and delivers something you can rely on. Get it wrong and you are months in, over budget, and arguing about what was agreed.

This guide walks you through how to evaluate agencies properly — without needing a technical background to do it.

Key Takeaways

- ✓ Most agency selection mistakes come from choosing on price or first impressions rather than a structured framework.
- ✓ A good agency asks thoughtful questions before quoting — speed at the proposal stage is not a positive sign.
- ✓ Code ownership, payment structure, and the process for handling changes are the three contract areas that matter most.
- ✓ Red flags are rarely deal-breakers on their own — but several together, or one that persists, deserves a direct conversation.
- ✓ Calling references (not just reading testimonials) is one of the most underused and effective steps in agency selection.
- ✓ An independent review of a proposal before you sign can surface issues that are easy to miss without a technical background.

Why agency selection goes wrong

Most people choose the wrong agency for one of three reasons.

They go on price alone. The cheapest quote wins. But a low quote often means a narrowly scoped estimate — one that grows significantly once the real work begins. By the time that becomes clear, you are already committed.

They go on first impressions. The agency had a nice website, a confident sales call, and a portfolio of work that looked impressive. But looking good and building well are different skills. Plenty of agencies are excellent at winning business and poor at delivering it.

They skip the awkward questions. Nobody wants to feel difficult in an early conversation with a potential supplier. So the hard questions — about timelines, about what happens when things go wrong, about who owns the code — do not get asked. And then they matter.

The good news is that none of this is complicated to avoid. It just requires a bit of structure and a willingness to treat this decision like any other significant purchase.

What to look for when evaluating an agency

When you are assessing an agency, ignore the surface-level stuff — the branding, the office location, the size of the team. Focus on these six things instead.

Evidence of similar work

Have they built something like what you need before? Not in the same industry necessarily, but a similar type of project — similar complexity, similar scale, similar kind of users. Ask to see it. Ask to speak to the client who commissioned it.

How they handle the brief

A good agency asks questions before they quote. If someone sends you a price within 24 hours of a first conversation, that is not efficiency — it is guesswork. You want an agency that slows down at the start, asks what you are trying to achieve, and comes back with a considered response.

How they communicate

You will be in a working relationship with these people for months. How do they communicate? Are they clear? Do they respond quickly? Do they explain things in plain language or do they hide behind jargon? The way an agency behaves before you hire them is usually the way they behave after.

What the contract actually says

Two things matter most in any development contract. First: who owns the code once it is built? It should be you. Second: what is the payment schedule tied to? It should be tied to milestones and deliverables, not just time. If a contract is vague on either of these, that is a problem.

Their process for handling change

Requirements change during a build. That is normal. What matters is whether the agency has a clear, fair way of dealing with it. Ask them directly: if we need to change something mid-project, how does that work? What gets agreed in writing? What does it cost? The answer tells you a lot.

References you can actually speak to

Not written testimonials on a website. Real clients you can call. Ask them: did the project come in on budget? Did it take as long as you were told? Would you use this agency again? Most agencies will provide references — but few buyers actually call them. Call them.

Red flags to watch for

Pause and ask questions if you see any of the following.

- They cannot show you a relevant example of previous work
- They quote a fixed price before fully understanding what you need
- They cannot clearly explain their process for managing the project
- They are vague about who owns the code at the end
- They discourage you from getting a second opinion
- They push for a long commitment before delivering anything
- They cannot name a single person who will be your main point of contact
- They make promises about timelines that seem too good to be true
- References are unavailable or only offered as written quotes
- Jargon is used to answer simple questions — clarity should never require a glossary

None of these are automatic disqualifiers, but each one deserves a direct conversation. If the answers do not satisfy you, trust that.

A simple scoring framework

When you have spoken to two or more agencies, use this framework to compare them side by side. Score each agency from 1 to 5 on each area. Add the scores up. It will not make the decision for you, but it will make the comparison honest.

What you are scoring	Agency A (1–5)	Agency B (1–5)
Relevant work they can show you		
Quality of questions they asked before quoting		
Clarity and speed of communication		
Contract terms — code ownership and payment		
Clear process for handling changes		
References you were able to speak to		
Overall confidence in the team		
TOTAL		

Print this out and fill it in by hand if that helps. The point is not to be scientific — it is to stop you making the decision based on whoever made the best impression in the sales call.

One question that changes the conversation

Ask every agency you speak to: 'What is the most common reason your projects go over budget or take longer than planned?' A good agency will answer this honestly and explain what they do to manage it. An agency that cannot answer it, or deflects, is telling you something important.

Downloadable vendor scorecard

The scoring table above is a starting point. The full vendor scorecard from Qube Catalyst gives you a more detailed evaluation tool — with guidance on how to weight each factor based on the type of project you are commissioning.

It covers:

- Evaluation criteria across seven areas
- Guidance on what a good and poor answer looks like for each
- A scoring summary you can share with a colleague or advisor
- A notes column for recording what each agency actually said

Not sure how to score the agencies you are talking to?

Qube Catalyst offers independent reviews of agency proposals and quotes — so you get an honest read on who to trust before you commit your budget. We have no stake in which agency you choose. Our only interest is making sure you choose the right one.

Learn more about independent proposal reviews from Qube Catalyst. **Book a call** or **Email me directly**

Getting an independent view

Even with a good framework, agency selection is hard when you do not have a technical background. You can ask the right questions and still not know whether the answers you are getting are honest.

This is where an independent review helps. Before you sign anything, having someone assess the proposal on your behalf — someone with no commercial interest in the outcome — can surface issues that are easy to miss.

Things an independent review typically catches:

- Quotes that are low because key elements have not been included in the scope
- Payment schedules that front-load your risk and back-load the agency's
- Contract terms that leave code ownership unclear
- Timelines that are unrealistic for the scope described
- Proposals that use impressive language to describe a generic approach

It is not about distrust. Most agencies are not trying to deceive you. But without independent input, you have no way of knowing what you do not know — and that is where the expensive surprises come from.

Key point

The agency that wins your business will have a significant amount of control over your money, your timeline, and your product. Spend the time to choose carefully. The cost of getting this wrong — in wasted budget, lost time, and a product that does not work — is far higher than the cost of doing the selection properly.

Quick checklist: before you sign with an agency

- I have spoken to at least two agencies and compared them directly
- I have seen relevant examples of their previous work
- I have spoken to at least one of their previous clients directly
- I have asked about their process for managing changes mid-project
- I understand who owns the code once the project is complete
- I know the payment schedule and what it is tied to
- I have a named point of contact for the project
- I have had the proposal reviewed by someone independent of the agency

Related reading

- How to Plan a Software Project as a Non-Technical Founder
- What a Realistic MVP Scope Looks Like
- How Founders Track Progress on a Software Build

Qube Catalyst helps business owners and founders plan, commission, and review software builds — independently of the developers doing the work.

Anchor page 3

What a Realistic MVP Scope Looks Like

If you are in the early stages of planning a software build, you have probably come across the term MVP. It gets used a lot — and it means very different things to different people. That confusion is responsible for a huge number of projects that run over budget, take twice as long as planned, or never launch at all.

After working on software projects for more than two decades, from large scale platforms with billions of users to small startups and early-stage builds — I have seen how often MVP scope quietly expands beyond what was originally intended

This guide explains what MVP actually means in practice, why scope so often grows beyond what was intended, and how to tell whether what you are building is genuinely focused or quietly becoming something much bigger.

If you are mid-build and things feel more complex than you expected, you are not alone. Scope creep is one of the most common experiences in software projects — and it is almost always fixable once you can see it clearly.

Key Takeaways

- ✓ MVP means the smallest version of your product that lets a real user do the most important thing it is supposed to do — nothing more.
- ✓ Scope creep almost always happens gradually, through a series of small additions that each seem reasonable on their own.
- ✓ The most common cause of scope growth is trying to serve every possible type of user rather than the most important one.
- ✓ A well-scoped MVP answers three questions: who is it for, what is the one thing it helps them do, and how will you know it is working?
- ✓ Launching with a focused product that works reliably is more valuable than a feature-rich one that is late and over budget.
- ✓ If your scope looks significantly different from when the project started, a scope review is worth doing before more development work is done.

What MVP actually means

MVP stands for Minimum Viable Product. It is a term borrowed from the startup world, but it applies just as well to a business upgrading a system or an individual commissioning their first app.

The simplest way to understand it is this: an MVP is the smallest version of your product that lets a real user do the most important thing it is supposed to do.

That is it. Not the smallest version of everything you want to build one day. Not a rough prototype. Not a half-finished product. A focused, working piece of software that does one core thing well enough to be genuinely useful.

The point of starting there is not to cut corners. It is to find out, quickly and at lower cost, whether what you are building actually works for the people it is intended for — before you have spent your entire budget on features that turn out not to matter.

A useful way to think about it: if someone used only your MVP and nothing else, would they get real value from it? If the answer is yes, the scope is probably right. If the answer is 'only if we also add X and Y and Z,' those additions need to be examined carefully.

> **A helpful way to think about it**
>
> Think of the first version of a product like a new member of staff on their first week. You would not expect them to do everything from day one. You would expect them to handle the most important tasks reliably, and grow from there. Your MVP works the same way.

Why MVP scope tends to grow

Scope creep — the gradual expansion of what a project is supposed to deliver — is extremely common. It rarely happens because someone makes a bad decision. It usually happens gradually, through a series of small, seemingly sensible additions.

Here is how it typically unfolds.

You start with a clear core idea. Then someone asks: 'What about users who want to do it this way instead?' So a second option gets added. Then: 'We should probably have a dashboard so people can see their history.' Then: 'It would be good to have email notifications.' Then: 'Can we add the ability to export to a spreadsheet?'

Each of these feels reasonable on its own. Taken together, they have doubled the scope — and with it, the cost, the timeline, and the complexity.

The most common causes of scope growth are:

- Trying to serve every possible type of user rather than the most important one
- Adding features to solve problems that may not exist yet
- Making assumptions about what users will want before testing with real ones
- Comparing the MVP to a mature competitor product with years of development behind it
- Discomfort with launching something that feels 'incomplete' — even when it isn't

That last one is important. The instinct to keep adding before launch is understandable. But a product that launches with a focused set of features and works reliably is more valuable than one with twice the features that is six months late and three times over budget.

What a healthy MVP includes

A well-scoped MVP has a clear answer to three questions:

- Who is this for?
- What is the one thing it helps them do?
- How will we know it is working?

Beyond that, the features in a healthy MVP tend to share a common characteristic: each one is either essential to the core user journey, or required for the product to work safely and reliably. Everything else is phase two.

A healthy MVP typically includes:

- The core user journey — the main thing the product exists to do, working end to end
- Basic account or access functionality, if users need to log in
- Enough visual design to feel trustworthy and easy to use — not polished to perfection
- Error handling — what happens when something goes wrong
- A way to collect feedback from early users
- Essential integrations only — the connections that are required, not the nice-to-haves

A healthy MVP usually does not include:

- Advanced reporting or analytics dashboards
- Multiple user roles with complex permissions
- Full customisation options for users
- Automated email sequences or marketing features
- Mobile apps (if a mobile-friendly website will do the same job)
- Integrations with tools that only a small number of users will need
- Admin panels with extensive management features
- Everything a competitor's mature product already has

This is not a rigid rule. Every project is different. But if your current scope includes most of what is in the amber list, it is worth asking whether those things are genuinely needed on day one — or whether they are there because they felt important at the time.

Signs your MVP is becoming a full product

It can be hard to notice scope creep while it is happening. These are some of the signs that a project intended as an MVP has grown into something much larger.

- The original timeline has been extended more than once to accommodate new requirements
- The feature list has grown significantly since the first brief was written
- There are now multiple types of users, each with different needs and journeys
- The project is waiting on decisions about features that were not in the original plan
- The cost estimate has increased, and the explanation involves 'additional requirements'
- Conversations with developers are getting more complex and harder to follow
- The launch date keeps moving because there is always one more thing to finish first

If several of these feel familiar, that is not a cause for alarm — it is a prompt to pause and reassess. A scope review at this stage, even a short one, can save a significant amount of time and money.

The question to ask is simply: what is the one thing this product must do for the first version to be worth launching? Everything else can be evaluated from there.

Not sure whether your current scope is realistic?

Qube Catalyst offers independent scope reviews — a straightforward conversation about what you are planning to build, whether the scope makes sense, and where the risks are. No jargon, no agenda. Just an honest outside view before you commit.

Learn more about independent scope reviews from Qube Catalyst. **Book a call** or **Email me directly**

A simple scope checklist

Use this checklist to sense-check your current scope. It is not a test to pass or fail — it is a set of questions to sit with honestly.

Question	Yes	Not sure
Can I describe the core thing this product does in one sentence?	☐	☐
Is there a single type of user this first version is primarily built for?	☐	☐
Could a real user get genuine value from this product without any of the extra features?	☐	☐
Does every feature on the list connect directly to the core user journey?	☐	☐
Is the scope roughly the same as when the project started?	☐	☐
Do I have a clear idea of what success looks like 90 days after launch?	☐	☐
Am I comfortable launching with this scope, rather than waiting to add more?	☐	☐

If several of your answers landed in the 'not sure' column, that is useful information. It suggests there are decisions still to be made — and making them now, before more development work is done, is almost always easier and cheaper than making them later.

A reassuring thought

Every founder and business owner who has commissioned a software build has found this part hard. Scoping is genuinely difficult — it asks you to make decisions about things you cannot fully see yet. The goal is not perfection. It is clarity. A clear, modest scope that you can build on is far more valuable than an ambitious one that gets stuck.

Related reading

- How to Plan a Software Project as a Non-Technical Founder
- How to Choose a Development Agency
- How Founders Track Progress on a Software Build

Qube Catalyst helps business owners and founders plan, commission, and review software builds — independently of the developers doing the work.

Anchor page 4

How to Track Progress on a Software Build

After working on software projects for more than two decades — from large platforms serving billions of users to early-stage builds — I've seen how often progress becomes hard for non-technical founders to judge during the middle of a project.

If you are mid-way through a software project and you are not quite sure how it is going, you are in good company. The period between signing with a developer and holding a finished product is one of the most uncertain experiences in business — especially when you do not have a technical background.

Updates arrive. Things get ticked off. Meetings happen. And yet somehow, progress feels invisible. You are spending money every week and it is difficult to know whether you are on track, ahead, or quietly drifting.

This guide gives you a straightforward way to follow what is happening in your project — without needing to understand the technical detail yourself.

Key Takeaways

- ✓ Feeling uncertain during a software build is normal — it does not mean something is wrong.
- ✓ Real progress means working software you can see and use, not just a list of tasks with ticks next to them.
- ✓ A weekly written update from your developer — received without having to ask — is the baseline standard to hold them to.
- ✓ Asking questions does not slow a project down; it usually surfaces small problems before they become large ones.
- ✓ Early signs of drift include vague updates, moving deadlines, and a growing gap between what is being built and what was originally agreed.
- ✓ If something feels wrong, getting an independent view early is far better than waiting until the problem is obvious.

Why progress becomes hard to see

Software is built in layers. A developer might spend two weeks building something that is entirely invisible to you — the plumbing behind the scenes that makes everything else work. From the outside, nothing appears to have changed. That can feel alarming, even when it is completely normal.

The challenge is that most developers are better at building software than they are at explaining it. Not because they are trying to be difficult — but because they live inside the detail of the work, and translating that into plain English takes time and effort that is easy to deprioritise.

At the same time, many non-technical clients are hesitant to ask too many questions, worried about appearing difficult or slowing the work down. So the gap between what is happening and what the client understands grows wider.

Neither side is at fault. But left unaddressed, this communication gap is one of the most reliable early indicators of a project that is heading off course.

Worth knowing

Asking questions does not slow a project down. In fact, regular short check-ins tend to surface small problems before they become large ones. A developer who is reluctant to explain what they are doing is a bigger concern than a client who asks too much.

What actually shows real progress

Not all progress updates are equal. A weekly message saying 'things are going well' is not a progress update. Neither is a long list of technical tasks with green ticks next to them — if you cannot interpret what those tasks mean, the list tells you nothing.

Real progress is visible and verifiable. Here is what to look for.

Working software you can see and use

The most reliable indicator of progress is something you can actually look at and interact with. It does not need to be finished or polished — but at regular intervals you should be able to see the product taking shape. If weeks go by without being shown anything, ask why.

Completed features against an agreed list

At the start of the project, or at the start of each phase, there should be an agreed list of what is being built. Progress means features from that list being completed and demonstrated to you — not just worked on, but finished well enough to show.

Clear answers to straightforward questions

If you ask 'where are we against the plan?' and the answer requires ten minutes of technical explanation, that is a problem. A project that is on track should be easy to summarise. 'We have completed X, we are working on Y, and Z comes next' is the kind of answer that shows someone has a clear grip on the work.

Honest reporting on anything that has slipped

Good developers flag problems early. If something is taking longer than planned, you should hear about it before it affects the timeline — not after. Proactive honesty about delays is a sign of a well-run project. Finding out about problems only when you ask is a warning sign.

Questions to ask your developer each week

You do not need to understand the technical detail to ask good questions. These are straightforward, reasonable questions that any developer running a well-organised project should be able to answer clearly.

#	Ask your developer this
1	What did you complete this week, and can you show me?
2	What are you working on next, and when do you expect it to be done?
3	Is anything taking longer than planned? If so, what and why?
4	Are there any decisions you need from me before you can move forward?
5	Are we still on track to hit the agreed milestone date?
6	Has anything come up that could affect the overall timeline or budget?

7	Is there anything I should know about that we have not discussed yet?

You do not need to ask all of these every week. But a short, regular check-in that covers most of them will give you a much clearer picture of where the project stands than a monthly update call ever will.

If the answers to these questions are consistently vague, delayed, or harder to follow than they should be, that is worth paying attention to.

What good reporting looks like

How a developer keeps you informed says a lot about how they manage their work. Good reporting does not need to be elaborate — but it should be consistent, clear, and arrive without you having to ask for it.

A simple, regular update

Once a week is the right frequency for most projects. The update should cover what was done, what is coming next, and anything that needs your attention. It should be written in plain language — not a list of technical terms you would need to look up.

A clear picture of where you are in the plan

You should always know, without having to ask, roughly where the project stands against the original timeline. Whether that is a simple checklist, a visual tracker, or a short summary, some form of 'here is where we are' should be part of every update.

Advance notice of anything changing

If a deadline is going to move, you should hear about it as soon as the developer knows — not on the day it was supposed to be delivered. The same applies to anything that might affect cost. No surprises is the standard to hold your developer to.

An open channel for your questions

You should have a clear way to reach your developer between formal updates — and a reasonable expectation of how quickly they will respond. For most projects, a response within one working day is a fair standard.

> **A useful benchmark**
>
> After reading a weekly update from your developer, you should be able to answer these three questions without asking a follow-up: What got done this week? What is happening next? Is the project on track? If you cannot, the update is not good enough — and it is entirely reasonable to say so.

Early signs a project may be drifting

Drift is gradual. Projects rarely go wrong all at once — they slip slowly, through a series of small changes that each seem manageable on their own. By the time the problem is obvious, a lot of time and money has often already been spent.

These are the early signals worth watching for. None of them alone means something is definitely wrong. But several of them together, or any one of them persisting over time, is worth taking seriously.

Watch for these signals

- Updates arrive late, or only when you chase for them
- Questions get long technical answers instead of straightforward ones
- Deadlines keep moving — each time with a reasonable-sounding explanation
- You are being shown less and less of the actual product as work progresses
- The scope has grown, but the timeline or budget has not been updated to reflect it
- Your developer seems less available than they were at the start of the project
- You feel like you need to prepare before conversations so you do not ask the wrong thing
- Your gut tells you something is off, even if you cannot point to exactly what

That last one matters. The people who commission software builds often notice something feels wrong before they can articulate what it is. If you have a persistent sense of unease about your project, it is worth getting an outside view — not to create a problem, but to either put your mind at rest or catch something early enough to fix it.

Feeling unsure about where your project actually stands?

Qube Catalyst offers independent project health checks — a clear-eyed look at where your build is, whether progress is real, and what, if anything, needs to change. We speak to you in plain English, with no agenda other than giving you an honest picture.

Learn more about independent project health checks from Qube Catalyst. **Book a call** or **Email me directly**

A simple weekly rhythm to keep things on track

You do not need a complicated system to stay on top of a software project. A consistent weekly rhythm is usually enough.

- Receive a written update from your developer every week, without having to ask
- Review it against your agreed list of features and milestones
- Note any questions and send them in one message rather than several
- Ask for a short call if anything is unclear or needs discussion
- Keep a brief record of what was agreed and when — a simple document or even an email thread is fine

The goal is not to micromanage the work. It is to stay informed enough that if something starts to slip, you notice it early — while there is still time and room to put it right.

A reassuring thought

Feeling uncertain during a software build is not a sign that something is wrong. It is a normal part of the process, especially if this is your first time commissioning one. The founders and business owners who come through it well are not the ones who understood everything — they are the ones who kept asking clear questions and stayed close enough to the project to notice when things needed attention.

Related reading

- What a Realistic MVP Scope Looks Like
- How to Choose a Development Agency
- How to Know if Your App Is Ready to Launch

Qube Catalyst helps business owners and founders plan, commission, and review software builds — independently of the developers doing the work.

Anchor page 5

How to Know if Your App Is Ready to Launch

You have spent months on this. The build is done — or your developer is telling you it is done. Everyone is pushing towards a launch date. And somewhere in the back of your mind, a quiet voice is asking: but is it actually ready?

That feeling is worth listening to. Not because something is necessarily wrong, but because launch is a significant moment — and approving a release without a proper check is one of the most common, and most costly, mistakes in software projects.

After more than two decades working on software projects — including large-scale platforms and early-stage products — I have seen how often founders are asked to approve launches they cannot realistically evaluate themselves.

This guide walks you through what launch readiness actually means, the checks worth doing before you go live, and what independent review looks like for people who want a second pair of eyes before they approve the release.

Key Takeaways

- ✓ Launch pressure is real — and it is exactly what makes honest readiness assessment difficult.
- ✓ Approving a release before the core user journey has been properly tested is the most common launch mistake.
- ✓ Seven areas need to be verified before any release: core journey, scope delivery, device testing, security basics, error handling, user support, and a rollback plan.
- ✓ Finding issues in a pre-launch review is a good outcome — it is always better to find them before users do.
- ✓ Most launch problems are not caused by bad development — they are caused by insufficient checking before the release was approved.
- ✓ An independent review gives you a written, plain-English assessment you can act on — not a technical report you cannot interpret.

Why launch readiness is hard to judge

There is a particular kind of pressure that builds as a project approaches launch. Everyone has been working towards this point. The developer wants to ship. You want to see the thing you have invested in finally go live. And the temptation to call it done and push it out is completely understandable.

But this pressure is exactly what makes honest launch readiness assessment difficult. It is hard to look clearly at something you are excited about, and harder still when the people around you are confident and keen to move forward.

There is also a knowledge gap to navigate. How are you supposed to judge whether the technical foundations are solid? Whether the product behaves correctly in situations your developer did not specifically test? Whether the thing you agreed to build is actually what has been built?

Without a technical background, most of this is invisible to you. And that is not a failure — it is just the reality of commissioning work you cannot fully evaluate yourself. The goal is not to evaluate every technical detail yourself, but to make sure someone who can has done so before you approve the release.

A useful way to think about it

You would not accept a building renovation without a snagging inspection. You would not sign off a legal document without a solicitor. Approving a software release is no different — it is a significant commitment, and an independent check before you make it is simply good practice.

The 7 checks before approving a release

These are the areas worth verifying before you give the go-ahead. You do not need to be technical to work through this list — but for some of these checks, you will want someone with the right experience to go deeper on your behalf.

#	Check	Yes	Not yet
1	The core user journey works end to end — a real user can do the main thing the product is supposed to do, without errors or confusion	☐	☐
2	Everything agreed in the original scope has been delivered — nothing significant is missing or deferred without your knowledge	☐	☐
3	The product has been tested on the devices and browsers your users are likely to use — not just the ones the developer happened to use during build	☐	☐
4	Basic security considerations have been addressed — particularly if the product handles personal data, payments, or user accounts	☐	☐
5	You know what happens when something goes wrong — errors are handled gracefully, and there is a plan for what to do if something breaks after launch	☐	☐
6	There is a way to support users after launch — even if that is just a contact email, people need somewhere to go if they run into problems	☐	☐
7	You have a rollback plan — if something goes seriously wrong in the first 48 hours, you know how to take the product offline or revert to a previous version	☐	☐

If you can tick all seven with confidence, you are in a strong position. If several of them land in the 'not yet' column, those gaps deserve attention before the launch date is confirmed — not after.

The goal is not to delay unnecessarily. It is to launch with confidence rather than hope.

Common launch mistakes — and how to avoid them

These are the patterns that come up most often when a launch does not go as planned. None of them are unusual, and all of them are avoidable.

Patterns worth watching for

- Launching before the core journey has been fully tested by someone other than the developer
- Approving a release because the deadline has arrived, rather than because the product is ready
- Assuming that 'it works on my laptop' means it works for everyone — devices, browsers, and internet speeds vary enormously
- Skipping the security basics because the product feels too small or low-profile to be a target
- Going live without a support plan — the first users to hit a problem need somewhere to go
- Not confirming that everything in the original scope has been delivered before approving the final payment
- Treating launch as the end of the project rather than the beginning of a new phase — what happens next matters as much as what happens now

Most of these mistakes happen not because someone was careless, but because the launch period is busy and pressured, and things get assumed rather than checked. A simple checklist and a calm hour before you approve the release will catch most of them.

What independent review looks like

An independent review — sometimes called a pre-launch review or sign-off check — is exactly what it sounds like. Someone who was not involved in building your product takes a structured look at it before it goes live, and gives you an honest assessment of where it stands.

It is not an audit. It is not an adversarial process. It is simply a second pair of eyes, from someone with no stake in the outcome, who can tell you clearly whether what you are about to release is ready.

What a good review covers

A thorough pre-launch review will work through the core user journeys — the main things your product is supposed to do — and check that they work correctly and consistently. It will compare what has been built against what was originally agreed, so you know whether the scope has been fully delivered. And it will look at the areas that are hardest for a non-technical client to evaluate: how the product behaves under different conditions, how errors are handled, and whether the basics of security and data handling have been addressed.

What you get from it

A clear written report in plain English. Not a list of technical findings you cannot interpret — a straightforward summary of what is working well, what needs attention before launch, and what can safely wait for a later version. Something you can read, understand, and act on.

When to do it

The right time for an independent review is when your developer tells you the product is ready — before you approve the final sign-off and release. That is the moment when it is most valuable, and still early enough to address anything that comes up without disrupting your plans significantly.

The Qube Catalyst Launch Sign-Off

Before you approve a release, we carry out an independent review of your product — testing the core user journeys, checking that what was agreed has been delivered, and giving you a clear written assessment of whether it is ready to go live.

You get a straightforward report — in plain English — that tells you what is working well, what needs attention before launch, and what can safely wait for a later version.

No jargon. No agenda. Just an honest, independent view from someone whose only interest is making sure your launch goes well.

Learn more about the Qube Catalyst Launch Sign-Off Review. **Book a call** or **Email me directly**

If you are not quite ready to launch

Sometimes a pre-launch review surfaces things that need attention. That is not a bad outcome — it is the point. Finding something before launch is almost always better than finding it after.

The most common findings in a pre-launch review fall into two categories:

- Things that should be fixed before launch — gaps in the core journey, security basics that have been skipped, significant features that were agreed but not delivered
- Things that are fine to address in a later version — nice-to-have improvements, minor polish, features that were always planned as phase two

A good review will tell you clearly which is which, so you can make an informed decision about when to launch — not a pressured one.

It is also worth knowing that finding issues at this stage is normal. Very few software products emerge from build without anything to address. The goal of a pre-launch review is not to find a perfect product — it is to make sure the product is good enough to launch confidently, with a clear plan for what comes next.

A reassuring thought

The founders and business owners who launch most successfully are not the ones who had perfect builds. They are the ones who were honest about where their product stood before they went live — and who made clear, calm decisions from that position rather than optimistic ones.

Before you approve the release — a final checklist

- The core user journey has been tested end to end by someone other than the developer
- Everything in the original scope has been delivered or knowingly deferred
- The product has been tested on the devices and browsers your users will actually use
- Basic security has been addressed, particularly around data and payments
- There is a plan for what happens when something goes wrong after launch
- Users have somewhere to go if they need help
- You have a rollback plan if things go seriously wrong in the first 48 hours
- You have had the product independently reviewed, or you are confident you understand what you are approving

You have put a significant amount of time, money, and energy into getting to this point. A careful final check before you launch is one of the best investments you can make in that effort.

Related reading

- How to Plan a Software Project as a Non-Technical Founder

- How Founders Track Progress on a Software Build
- What a Realistic MVP Scope Looks Like

Qube Catalyst helps business owners and founders plan, commission, and review software builds — independently of the developers doing the work.

Central hub page

Qube Catalyst
Practical guidance for business owners and founders commissioning a software build

A Practical Guide for Non-Technical Founders Commissioning Software

Commissioning a software build — whether you are a business owner upgrading a system or an individual turning an idea into an app — is one of the most complex things you can do without a technical background.

The guidance here is based on more than two decades of experience working on software products, including eight years inside Meta and many years working with startups and founders who are not technical commissioning their first platforms.

There is a lot to navigate. You are making significant decisions about money, time, and risk, often while working with people who speak a different professional language. Most of the guidance out there is written for developers or product managers, not for the people actually paying for the work.

This library of guides was written specifically for you. It covers the full journey — from the first planning conversations through to the moment you decide whether your product is ready to go live. Each guide is written in plain English, with no assumed knowledge, and with one goal: to help you make good decisions at every stage.

How to use this page

If you are just starting out, read the guides in order — they follow the natural shape of a software project from beginning to end. If you are already mid-build, find the stage that matches where you are now and start there. Each guide also links to the others, so you can move between them as your project progresses.

Find the guide for where you are right now

Not sure which guide to start with? This table maps each stage of a software project to the guide that will help most.

Stage	Where you are	Start with this guide
1	You have an idea or a problem to solve — and you are not sure where to begin	**How to Plan a Software Project**
2	You are ready to find a developer or agency and want to choose the right one	**How to Choose a Development Agency**
3	You are scoping the build and want to make sure it stays focused and deliverable	**What a Realistic MVP Scope Looks Like**
4	Development is underway and you want to stay properly informed	**How to Track Progress on a Software Build**
5	Your developer says it is ready — and you want to be sure before you approve	**How to Know if Your App Is Ready to Launch**

The five guides

Each guide below covers one stage of the journey in full. They are designed to be read independently, but they work best as a complete set.

Stage 1 **Idea and preparation**

How to Plan a Software Project When You Are Not Technical

Before you speak to a single developer, there is thinking to do. This guide walks you through what you need to get clear on — whether you are building something new or replacing a system you already use — so that when you do start talking to people, you are having the right conversations.

What this guide covers:

- Why so many projects run into trouble before they have even started
- The five things to settle before you approach anyone
- A simple planning framework you can fill in before any scoping conversation
- What business owners upgrading existing systems need to think about first

Read the guide: **qubecatalyst.com/how-to-plan-a-software-project**

Stage 2 **Selecting the right partner**

How to Choose a Development Agency

Choosing who builds your product is one of the most consequential decisions in the whole project. This guide gives you a clear framework for evaluating agencies — without needing any technical knowledge to do it properly.

What this guide covers:

- Why agency selection so often goes wrong — and how to avoid the common traps
- Six things that actually matter when comparing agencies
- Red flags worth watching for before you sign anything
- A simple scoring framework for comparing agencies side by side

Read the guide: **qubecatalyst.com/how-to-choose-a-development-agency**

Stage 3 **Defining what gets built**

What a Realistic MVP Scope Looks Like

One of the most reliable ways to overspend on a software project is to build too much too soon. This guide explains what MVP actually means in practice, why scope has a habit of growing, and how to tell whether your current plans are focused or quietly becoming something much larger.

What this guide covers:

- What MVP means — and what it does not
- Why scope grows even when nobody intends it to
- What a well-focused first version typically includes (and what can wait)
- Signs that your scope has grown beyond what the first version needs

Read the guide: **qubecatalyst.com/what-a-realistic-mvp-scope-looks-like**

Stage 4 **During development**

How to Track Progress on a Software Build

Development is the part of the project where non-technical clients most often feel lost. Progress seems invisible. Updates are hard to interpret. This guide gives you a straightforward way to stay informed — and the questions to ask each week to make sure things are actually moving in the right direction.

What this guide covers:

- Why progress is so hard to see — and why that is normal
- What real progress actually looks like (as opposed to busy-looking updates)
- Seven questions to ask your developer every week
- Early signs that a project may be starting to drift

Read the guide: **qubecatalyst.com/how-to-track-progress-on-a-software-build**

Stage 5 **Before you go live**

How to Know if Your App Is Ready to Launch

When your developer says it is ready, how do you know whether to believe them? This guide takes you through the checks worth making before you approve a release — and what independent review looks like for people who want a second opinion before they commit.

What this guide covers:

- Why launch readiness is genuinely difficult to judge without technical knowledge
- The seven checks before approving a release
- Common launch mistakes — and how to avoid them
- What a pre-launch independent review involves and what you get from it

Read the guide: **qubecatalyst.com/how-to-know-if-your-app-is-ready-to-launch**

Free tools and templates

Alongside the guides, Qube Catalyst publishes free tools you can download and use in your project. These are practical working resources — scorecards, checklists, and templates — designed to make the guidance in the guides easier to act on.

Free tool **Agency evaluation**

Development Agency Selection Scorecard

A structured Excel workbook for comparing development agencies side by side. Eight evaluation categories, automatic scoring totals, explanations of each factor, a scoring guide, and a notes sheet with the ten questions worth asking every agency you speak to.

What it includes:

- **Scoring sheet — rate each agency 1–5 across eight categories, totals calculate automatically**
- **Factor explanations — plain-English guidance on what each category is asking you to judge**
- **Scoring guide — what a score of 1 through 5 looks like in practice**
- **Notes sheet — ten questions to ask, with space to record exactly what each agency said**

Download free: qubecatalyst.com/agency-scorecard

Related guide: How to Choose a Development Agency

About Qube Catalyst

These guides are written and published by Qube Catalyst. We work with business owners and founders who are commissioning software builds — providing independent guidance and review at every stage of the process.

Independent is the important word. We are not developers. We have no commercial interest in which agency you choose or whether a build goes ahead. Our only interest is making sure that when you commit your time and money to a software project, you do so with a clear picture of what you are getting into — and the best possible chance of a good outcome.

About the author

Pearl Oguchi

Pearl has worked in technology for more than 20 years, including eight years at Meta. She now helps business owners and founders commission, review, and launch software products with confidence — providing the independent perspective that most software projects are missing.

Work with Qube Catalyst

The guides on this page give you the framework. If you would like an independent expert alongside you at any stage — from planning through to launch — that is what Qube Catalyst is here for.

We help with:

- Scoping and planning — getting clear on what you need before you commission anything
- Proposal and contract review — an honest read on what a developer is offering before you sign
- Progress monitoring — independent check-ins during a build to make sure things are on track
- Pre-launch sign-off — a structured review of your product before it goes live

Get in touch: **qubecatalyst.com**

All guides in this library

- How to Plan a Software Project When You Are Not Technical
- How to Choose a Development Agency
- What a Realistic MVP Scope Looks Like
- How to Track Progress on a Software Build
- How to Know if Your App Is Ready to Launch

Free tools

- Development Agency Selection Scorecard — free download at qubecatalyst.com/agency-scorecard

Qube Catalyst helps business owners and founders plan, commission, and review software builds — independently of the developers doing the work.